Part III

Non-Standard Languages

9 Introduction

9.1 Kind of definition

First, we may distinguish between systems whose languages (i.e. sets of wffs) are **defined recursively** and those whose languages are **defined explicitly**.

This distinction is not common, since formal languages used in logic are normally defined recursively.

However, as we previously saw, the formal language of Aristotelian syllogistic can be defined as a finite set of categorical propositions. Thus, it is basically syllogistic languages vs modern logical languages.

Non-Standard Logical Symbols 10

Let us consider some non-standard logical symbols. That is, symbols featured in formal languages used in logic, which are not part of standard languages.

10.1 Logical value constants

Some languages include constant for logical values. For instance, it is not uncommon to have languages with '$\mathbf{1}$' as a propositional constant for representing *truth* and '$\mathbf{0}$' for *falsity*.

We may define one such enhancement S of $\mathsf{S0}$ as follows:

Preliminary 10.1 (Formulae with logical value constants)**.** The set $\mathbf{Fo}_{\mathsf{S}}$ is recursively defined as follows:

1. $\mathsf{p} \in \mathbf{Non}_{\mathsf{S}} \cup \{\text{‘}\mathbf{0}\text{’}, \text{‘}\mathbf{1}\text{’}\} \Rightarrow \mathsf{p} \in \mathbf{X}$.
2. $\mathsf{p} \in \mathbf{X} \Rightarrow \ulcorner\neg\mathsf{p}\urcorner \in \mathbf{X}$.
3. $\mathsf{p}, \mathsf{q} \in \mathbf{X} \Rightarrow \ulcorner(\mathsf{p} \vee \mathsf{q})\urcorner, \ulcorner(\mathsf{p} \wedge \mathsf{q})\urcorner, \ulcorner(\mathsf{p} \rightarrow \mathsf{q})\urcorner, \ulcorner(\mathsf{p} \leftrightarrow \mathsf{q})\urcorner \in \mathbf{X}$.
4. $\mathbf{Fo}_{\mathsf{S}}$ is the intersection of all sets $\mathbf{X}$ satisfying 1–3.

We might define '$\mathbf{Fo}_{\mathsf{S}}$' more shortly using '$\overset{\text{DF}}{=}$'.

Definition 10.2 (Formulae with logical value constants)**.**

$$\begin{aligned}\mathbf{Fo}_{\mathsf{S}} \overset{\text{DF}}{=} \mathbf{Non}_{\mathsf{S}} &\cup \{\text{‘}\mathbf{0}\text{’}, \text{‘}\mathbf{1}\text{’}\} \cup \\ &\{\ulcorner\bullet\mathsf{p}\urcorner : \mathsf{p} \in \mathbf{Fo}_{\mathsf{S}} \mathrel{\wedge\!\!\!\wedge} \bullet \in \{\text{‘}\neg\text{’}, \text{‘}\Box\text{’}, \text{‘}\Diamond\text{’}\}\} \cup \\ &\{\ulcorner(\mathsf{p} \heartsuit \mathsf{q})\urcorner : \mathsf{p}, \mathsf{q} \in \mathbf{Fo}_{\mathsf{S}} \mathrel{\wedge\!\!\!\wedge} \heartsuit \in \mathbf{Con}\}.\end{aligned}$$

The sense of an expression featuring these kinds of logical symbols is difficult to translate into natural language. The following are examples, two of which are expressed in a language different to that defined in definition 10.2.

1. If p, then the true.
   Representation: '$(p \rightarrow \mathbf{1})$'.
2. If the false, then I sing like Freddy Mercury.
   Representation: '$(\mathbf{0} \rightarrow q)$'.
3. If every proposition is true, then every proposition is false.
   Representation 1: '$((p \leftrightarrow \mathbf{0}) \rightarrow (p \leftrightarrow \mathbf{1}))$'.
   Representation 2: '$\forall p\,(p = \mathbf{0} \rightarrow p = \mathbf{1})$'.
   Representation 3: '$(\forall p\,(p = \mathbf{0}) \rightarrow \forall p\,(p = \mathbf{1}))$'.

This may also be done of any other logical value used for constructing interpretation in for a logical system.

10.2 Modal operators

Modal logic is nowadays a standard extension of classical logic, providing formal tools to reason about notions such as necessity, possibility, knowledge, time, obligation, and location.

There is a wide variety of modal systems, each motivated by different applications, and often equipped with distinct symbols. Below we present a non-exhaustive overview of some of the most commonly used modal operators and their intended readings.

10.2.1 Alethic Modality

We may speak about possibilities, contingencies, necessities, impossibilities, and the like, in modal languages. We commonly use '$\Box$' for expressing necessity and '$\Diamond$' for possibility. A modal 0th-order formal language $\mathbf{Fo}_S$ may be constructed as follows.

Preliminary 10.3 (Formulae of propositional modal logic)**.** The set $\mathbf{Fo}_S$ is recursively defined as follows:

1. $p \in \mathbf{Non}_S \Rightarrow p \in \mathbf{X}$.
2. $p \in \mathbf{X} \Rightarrow \ulcorner \neg p \urcorner, \ulcorner \Box p \urcorner, \ulcorner \Diamond p \urcorner \in \mathbf{X}$.
3. $p, q \in \mathbf{X} \Rightarrow \ulcorner (p \vee q) \urcorner, \ulcorner (p \wedge q) \urcorner, \ulcorner (p \rightarrow q) \urcorner, \ulcorner (p \leftrightarrow q) \urcorner \in \mathbf{X}$.
4. $\mathbf{Fo}_S$ is the intersection of all sets $\mathbf{X}$ satisfying 1–3.

We might define '$\mathbf{Fo}_S$' more shortly using '$\overset{\mathrm{DF}}{=}$'.

Definition 10.4 (Formulae of propositional modal logic)**.**

$$\begin{aligned} \mathbf{Fo}_S \overset{\mathrm{DF}}{=} \ & \mathbf{Non}_S \cup \\ & \{\ulcorner \bullet p \urcorner : p \in \mathbf{Fo}_S \ \&\!\!\& \ \bullet \in \{`\neg', `\Box', `\Diamond'\}\} \cup \\ & \{\ulcorner (p \heartsuit q) \urcorner : p, q \in \mathbf{Fo}_S \ \&\!\!\& \ \heartsuit \in \mathbf{Con}\}. \end{aligned}$$

Table 10.1: Alethic modal formulae and their natural-language interpretations.

Type	Formula	Natural language sentence
Necessity	$\Box p$	It is necessary that Peirce is a logician. It is necessarily the case that Peirce is a logician. In all possible worlds, Peirce is a logician.
Possibility	$\Diamond p$	It is possible that Peirce is a logician. It is possibly the case that Peirce is a logician. In some possible world, Peirce is a logician.
Impossibility	$\neg\Diamond p$	It is not possible that Peirce is a logician. It is impossible that Peirce is a logician.
Contingency	$(\Diamond p \wedge \Diamond\neg p)$	It is contingent that Peirce is a logician.

The informal sense of usual modal expressions is summarised in Table 10.1.

Task 10.1. Formalise the following modal expressions.

1. Necessarily, either it rains now or it does not rain now.
2. Either it necessarily rains now or it necessarily does not rain now.
3. It is necessary that, if you are a bachelor, then you are unmarried.
4. If you are necessarily human, then you are possibly an animal.

Solution to task 10.1.

1. Necessarily, either it rains now or it does not rain now.

   Representation: '$\Box(r \vee \neg r)$'.
2. Either it necessarily rains now or it necessarily does not rain now.

   Representation: '$(\Box r \vee \Box\neg r)$'.
3. It is necessary that, if you are a bachelor, then you are unmarried.

   Representation: '$\Box(p \rightarrow q)$'.
4. If you are necessarily an animal, then you are possibly human.

   Representation: '$(\Box p \rightarrow \Diamond q)$'.

It is important to note that '$\Box$' and '$\Diamond$' may also be used in non-alethic modal senses, which include epistemic, deontic, and temporal senses, among others.

10.2.2 Epistemic modality

Epistemic expressions serve to express that a given agent knows, believes, or has some epistemic attitude towards some statement.

An epistemic language may use a modal epistemic operator 'K', so that a modal 0th-order epistemic language $\mathbf{Fo}_S$ may be constructed as follows.

Preliminary 10.5 (Formulae of propositional epistemic logic)**.** The set $\mathbf{Fo}_S$ is recursively defined as follows:

1. $p \in \mathbf{Non}_S \Rightarrow p \in \mathbf{X}$.
2. $p \in \mathbf{X} \Rightarrow \ulcorner\neg p\urcorner, \ulcorner Kp\urcorner \in \mathbf{X}$.
3. $p, q \in \mathbf{X} \Rightarrow \ulcorner(p \vee q)\urcorner, \ulcorner(p \wedge q)\urcorner, \ulcorner(p \rightarrow q)\urcorner, \ulcorner(p \leftrightarrow q)\urcorner \in \mathbf{X}$.
4. $\mathbf{Fo}_S$ is the intersection of all sets $\mathbf{X}$ satisfying 1–3.

We might define '$\mathbf{Fo}_S$' more shortly using '$\overset{\text{DF}}{=}$'.

Definition 10.6 (Formulae of propositional epistemic logic)**.**

$$\begin{aligned}\mathbf{Fo}_S \overset{\text{DF}}{=} \mathbf{Non}_S \cup \\ &\{\ulcorner \odot p\urcorner : p \in \mathbf{Fo}_S \And \odot \in \{\text{`}\neg\text{'}, \text{`}K\text{'}\}\} \cup \\ &\{\ulcorner(p \heartsuit q)\urcorner : p, q \in \mathbf{Fo}_S \And \heartsuit \in \mathbf{Con}\}.\end{aligned}$$

The informal sense of usual modal expressions is summarised in Table 10.2.

Task 10.2. Formalise the following epistemic expressions.

1. Either we know that it rains now or we do not know that it rains now.
2. Either we know that it rains now or we do know that it does not rains now.
3. We know that, if it God is known to exist, then life has a divine origin.
4. We know that, if it God is known to exist, then we know that life has a divine origin.

Table 10.2: Epistemic formulae and their natural-language interpretations.

Type	Formula	Natural language sentence
Positive knowledge	$\mathsf{K}p$	It is known that Peirce is a logician. We know that Peirce is a logician.
Negative knowledge	$\mathsf{K}\neg p$	It is known that Peirce is not a logician. We know that Peirce is not a logician.
Ignorance	$\neg\mathsf{K}p$	It is not known that Peirce is a logician. We ignore if Peirce is a logician (he may be or may not be).

Solution to task 10.2.

1. Either we know that it rains now or we do not know that it rains now.

   Representation: '$(\mathsf{K}r \vee \neg\mathsf{K}r)$'.

2. Either we know that it rains now or we do know that it does not rains now.

   Representation: '$(\mathsf{K}r \vee \mathsf{K}\neg r)$'.

3. We know that, if it God is known to exist, then life has a divine origin.

   Representation: '$\mathsf{K}(\mathsf{K}p \rightarrow q)$'.

4. We know that, if it God is known to exist, then we know that life has a divine origin.

   Representation: '$\mathsf{K}(\mathsf{K}p \rightarrow \mathsf{K}q)$'.

Other kinds of epistemic attitudes, like belief or entertainment, may expressed with similar languages.

Moreover, we may index an epistemic operator, so that '$\mathsf{K}_a p$' means that agent a knows that p.

10.2.3 Deontic Modality

Deontic expressions serve to express obligations or permission in relation to the state of affairs denoted by some statement.

An deontic language may use a modal deontic operators ‘O’, ‘P’, so that a modal 0th-order deontic language $\mathbf{Fo}_{\mathrm{S}}$ may be constructed as follows.

Preliminary 10.7 (Formulae of propositional deontic logic)**.** The set $\mathbf{Fo}_{\mathrm{S}}$ is recursively defined as follows:

1. $\mathrm{p} \in \mathbf{Non}_{\mathrm{S}} \Rightarrow \mathrm{p} \in \mathbf{X}$.
2. $\mathrm{p} \in \mathbf{X} \Rightarrow \ulcorner\neg\mathrm{p}\urcorner, \ulcorner\mathrm{Op}\urcorner, \ulcorner\mathrm{Pp}\urcorner \in \mathbf{X}$.
3. $\mathrm{p}, \mathrm{q} \in \mathbf{X} \Rightarrow \ulcorner(\mathrm{p} \vee \mathrm{q})\urcorner, \ulcorner(\mathrm{p} \wedge \mathrm{q})\urcorner, \ulcorner(\mathrm{p} \rightarrow \mathrm{q})\urcorner, \ulcorner(\mathrm{p} \leftrightarrow \mathrm{q})\urcorner \in \mathbf{X}$.
4. $\mathbf{Fo}_{\mathrm{S}}$ is the intersection of all sets $\mathbf{X}$ satisfying 1–3.

We might define ‘$\mathbf{Fo}_{\mathrm{S}}$’ more shortly using ‘$\overset{\mathrm{DF}}{=}$’.

Definition 10.8 (Formulae of propositional deontic logic)**.**

$$\begin{aligned}\mathbf{Fo}_{\mathrm{S}} \overset{\mathrm{DF}}{=} \mathbf{Non}_{\mathrm{S}} &\cup \\ &\{\ulcorner\bullet\mathrm{p}\urcorner : \mathrm{p} \in \mathbf{Fo}_{\mathrm{S}} \mathrel{/\!\!\!\wedge} \bullet \in \{‘\neg’, ‘\mathrm{O}’, ‘\mathrm{P}’\}\} \cup \\ &\{\ulcorner(\mathrm{p} \heartsuit \mathrm{q})\urcorner : \mathrm{p}, \mathrm{q} \in \mathbf{Fo}_{\mathrm{S}} \mathrel{/\!\!\!\wedge} \heartsuit \in \mathbf{Con}\}.\end{aligned}$$

The informal sense of usual modal expressions is summarised in Table 10.3.

Task 10.3. Formalise the following deontic expressions.

1. It is permitted both to stay and not to stay.
2. It is obligatory both to stay and not to stay.
3. If it is not obligatory to stay, then it is permitted not to stay.
4. It is mandatory that it be permitted to stay.

Table 10.3: Epistemic formulae and their natural-language interpretations.

Type	Formula	Natural language sentence
Obligation	$\mathsf{O}p$	It is obligatory that the door be open.
		It is mandatory that the door be open.
	$\mathsf{O}\neg p$	It is obligatory that the door be not open.
	$\neg\mathsf{O}p$	It is not obligatory that the door be open
Permission	$\mathsf{P}p$	It is permitted that the door be open.
	$\mathsf{P}\neg p$	It is permitted that the door be not open.
	$\neg\mathsf{P}p$	It is not permitted that the door be open.

Solution to task 10.3.

1. It is permitted to stay and it is permitted not to stay.

   Representation: '$(\mathsf{P}p \wedge \mathsf{P}\neg p)$'.
2. It is obligatory both to stay and not to stay.

   Representation: '$\mathsf{O}(p \wedge \neg p)$'.
3. If it is not obligatory to stay, then it is permitted not to stay.

   Representation: '$(\neg\mathsf{O}p \to \mathsf{P}\neg p)$'.
4. It is mandatory that it be permitted to stay and not to stay.

   Representation: '$\mathsf{OP}(p \wedge \neg p)$'.

10.2.4 Temporal Modality

Temporal or tense-relative expressions serve to express statements in relation to time.

A temporal or tense language may use a modal tense operators 'G', 'H', so that a modal 0th-order temporal language $\mathbf{Fo}_S$ may be constructed as follows.

Preliminary 10.9 (Formulae of propositional temporal logic)**.** The set $\mathbf{Fo}_S$ is recursively defined as follows:

1. $p \in \mathbf{Non}_S \Rightarrow p \in \mathbf{X}$.
2. $p \in \mathbf{X} \Rightarrow \ulcorner\neg p\urcorner, \ulcorner \mathsf{G}p\urcorner, \ulcorner \mathsf{H}p\urcorner \in \mathbf{X}$.
3. $p, q \in \mathbf{X} \Rightarrow \ulcorner(p \vee q)\urcorner, \ulcorner(p \wedge q)\urcorner, \ulcorner(p \longrightarrow q)\urcorner, \ulcorner(p \leftrightarrow q)\urcorner \in \mathbf{X}$.
4. $\mathbf{Fo}_S$ is the intersection of all sets $\mathbf{X}$ satisfying 1–3.

We might define '$\mathbf{Fo}_S$' more shortly using '$\overset{\text{DF}}{=}$'.

Definition 10.10 (Formulae of propositional temporal logic)**.**

$$\begin{aligned}\mathbf{Fo}_S \overset{\text{DF}}{=} \mathbf{Non}_S \cup &\\ &\{\ulcorner \boldsymbol{\odot} p\urcorner : p \in \mathbf{Fo}_S \mathrel{/\!\!\!\wedge\!\!\!\wedge} \boldsymbol{\odot} \in \{`\neg\text{'}, `\mathsf{G}\text{'}, `\mathsf{H}\text{'}\}\}\cup\\ &\{\ulcorner(p \heartsuit q)\urcorner : p, q \in \mathbf{Fo}_S \mathrel{/\!\!\!\wedge\!\!\!\wedge} \heartsuit \in \mathbf{Con}\}.\end{aligned}$$

The informal sense of usual modal expressions is summarised in Table 10.4.

Task 10.4. Formalise the following temporal expressions.

1. Peter was German and will always be German.
2. Peter was French, but will no longer be.
3. If you were a comedian, you will always be a comedian..
4. If George is French, then he will always be.

Table 10.4: Temporal formulae and their natural-language interpretations.

Type	Formula	Natural language sentence
Future	$\mathsf{G}p$	It always will be the case, that the door is open.
	$\neg\mathsf{G}\neg p$	Not always will be the case, that the door is not open. It will be the case that the door is open.
Past	$\mathsf{H}p$	It always was the case, that the the door is open.
	$\neg\mathsf{H}\neg p$	Not always was the case, that the door is not open. It was the case that the door is open.

Solution to task 10.4.

1. Peter was German and will always be German.

   Representation: '$(\neg\mathsf{H}\neg p \wedge \mathsf{G}p)$'.

2. Peter was French, but will no longer be.

   Representation: '$(\neg\mathsf{H}\neg p \wedge \mathsf{G}\neg p)$'.

3. If you were always a comedian, you will be a comedian.

   Representation: '$(\mathsf{H}p \longrightarrow \neg\mathsf{G}\neg p)$'.

4. If George is French, then he will always be.

   Representation: '$(p \longrightarrow \mathsf{G}p)$'.

10.2.5 Remark

A given modal operator may receive different semantic interpretations depending on the underlying logical framework (e.g. Kripke semantics, neighbourhood semantics, topological semantics), and different modal logics may validate different principles. Consequently, modal operators should always be understood relative to the specific system and application in which they are employed.

10.3 Term-forming symbols

In this category we have several symbols. The following are the most usual examples.

10.3.1 The definite descriptor

A definite description is an expression whose intended referent is the unique entity satisfying some property. We do so in the context of a first-order language, and construct singular terms with 'ι' of the form $\ulcorner\iota x F\urcorner$.

For our vocabularies, we will use those of S1. Thus, a first-order language $\mathbf{Fo}_\mathsf{S}$ with a definite descriptor with 'ι' may be constructed as follows.

Preliminary 10.11 (Formulae with definite descriptors)**.** Where $n \in \mathbf{Num}_\mathbb{W}$, the set $\mathbf{Fo}_\mathsf{S}$ is recursively defined as follows:

1. $t \in \mathbf{ICon}_{\mathsf{S1}} \cup \mathbf{IVar}_{\mathsf{S1}} \Rightarrow t \in \mathbf{Ter}_\mathsf{S}$.
2. $F \in \mathbf{X} \Rightarrow \ulcorner\neg F\urcorner \in \mathbf{X}$.
3. $F, G \in \mathbf{X} \Rightarrow \ulcorner(F \vee G)\urcorner, \ulcorner(F \wedge G)\urcorner, \ulcorner(F \rightarrow G)\urcorner, \ulcorner(F \leftrightarrow G)\urcorner \in \mathbf{X}$.
4. $F \in \mathbf{X} \curlywedge x \in \mathbf{IVar}_{\mathsf{S1}} \Rightarrow \ulcorner\iota x F\urcorner \in \mathbf{Ter}_\mathsf{S}$.
5. $P \in \mathbf{Pre}_n \curlywedge t_1, ..., t_n \in \mathbf{Ter}_\mathsf{S} \Rightarrow Pt_1...t_n \in \mathbf{X} : n \in \mathbb{N}$.
6. $F \in \mathbf{X} \curlywedge x \in \mathbf{IVar}_{\mathsf{S1}} \Rightarrow \ulcorner\exists x\, F\urcorner, \ulcorner\forall x\, F\urcorner \in \mathbf{X}$.
7. $\mathbf{Fo}_\mathsf{S}$ is the intersection of all sets $\mathbf{X}$ satisfying 1–6.

As with '$\mathbf{Fo}_{\mathsf{S0}}$', we might define '$\mathbf{Fo}_\mathsf{S}$' more shortly using '$\stackrel{\text{DF}}{=}$'.

Definition 10.12 (Formulae with definite descriptors)**.**

$$\mathbf{Ter}_\mathsf{S} \stackrel{\text{DF}}{=} \mathbf{ICon}_{\mathsf{S1}} \cup \mathbf{IVar}_{\mathsf{S1}} \cup \{\ulcorner\iota x F\urcorner : F \in \mathbf{Fo}_\mathsf{S} \curlywedge x \in \mathbf{IVar}_{\mathsf{S1}}\}$$

$$\begin{aligned}\mathbf{Fo}_\mathsf{S} \stackrel{\text{DF}}{=} & \{Pt_1...t_n : P \in \mathbf{Pre}_\mathsf{S}[n] \curlywedge t_1...t_n \in \mathbf{Ter}_\mathsf{S}\} \cup \\ & \{\ulcorner\neg F\urcorner : F \in \mathbf{Fo}_\mathsf{S}\} \cup \\ & \{\ulcorner(F \heartsuit G)\urcorner : F, G \in \mathbf{Fo}_\mathsf{S} \curlywedge \heartsuit \in \mathbf{Con}_{\mathsf{S1}}[2]\} \cup \\ & \{\bullet x F : \bullet \in \mathbf{Qua}_{\mathsf{S1}} \curlywedge x \in \mathbf{IVar}_{\mathsf{S1}} \curlywedge F \in \mathbf{Fo}_\mathsf{S}\} \\ & : n \in \mathbb{N}\end{aligned}$$

An expression like '$\iota x A(x)$' denotes the unique x such that $A(x)$. However, this expression is not a formula: it cannot be interpreted as true or false.

Task 10.5. Provide natural language expressions with the structure of the following formulae with definite descriptors:

1. '$Q^1 \iota x P x$'. **Regimented:** '$Q^1(\iota x P(x))$'.
2. '$R^2 \iota x P^2 x a b$'. **Regimented:** '$R^2(\iota x P^2(x, a), b)$'.
3. '$\neg P^1 \iota x S^2 x b$'. **Regimented:** '$\neg P^1(\iota x S^2(x, b))$'.
4. '$R^2 a \iota x P^1 x$'. **Regimented:** '$R^2(a, \iota x P^1(x))$'.

Solution to task 10.5.

1. '$Q^1 \iota x P x$'. **Regimented:** '$Q^1(\iota x P(x))$'.

   Natural language expression:

   The king of France is bald.
2. '$R^2 \iota x P^2 x a b$'. **Regimented:** '$R^2(\iota x P^2(x, a), b)$'.

   Natural language expression:

   The editor of *Mind* rejected the paper.
3. '$\neg P^1 \iota x S^2 x b$'. **Regimented:** '$\neg P^1(\iota x S^2(x, b))$'.

   Natural language expression:

   Bertha's unique sister is not Prussian.
4. '$R^2 a \iota x P^1 x$'. **Regimented:** '$R^2(a, \iota x P^1(x))$'.

   Natural language expression:

   Ana loves the author of *Hamlet*.

10.3.2 The indefinite descriptor

An indefinite description is an expression whose intended referent is some (not necessarily unique) entity satisfying a given property, if there is one. We introduce such expressions in a first-order language by means of Hilbert's ε-operator, which forms singular terms with 'ε' of the form $\ulcorner \varepsilon \mathtt{x} \mathtt{F} \urcorner$.

As before, we work with the vocabulary of S1. Thus, a first-order language $\mathbf{Fo}_{\mathsf{S}}$ with an indefinite descriptor using ε may be constructed as follows.

Preliminary 10.13 (Formulae with indefinite descriptors)**.** Where $\mathtt{n} \in \mathbf{Num}_{\mathbb{W}}$, the set $\mathbf{Fo}_{\mathsf{S}}$ is recursively defined as follows:

1. $\mathtt{t} \in \mathbf{ICon}_{\mathsf{S1}} \cup \mathbf{IVar}_{\mathsf{S1}} \Rightarrow \mathtt{t} \in \mathbf{Ter}_{\mathsf{S}}$.
2. $\mathtt{F} \in \mathbf{X} \Rightarrow \ulcorner \neg \mathtt{F} \urcorner \in \mathbf{X}$.
3. $\mathtt{F}, \mathtt{G} \in \mathbf{X} \Rightarrow \ulcorner (\mathtt{F} \vee \mathtt{G}) \urcorner, \ulcorner (\mathtt{F} \wedge \mathtt{G}) \urcorner, \ulcorner (\mathtt{F} \rightarrow \mathtt{G}) \urcorner, \ulcorner (\mathtt{F} \leftrightarrow \mathtt{G}) \urcorner \in \mathbf{X}$.
4. $\mathtt{F} \in \mathbf{X} \curlywedge \mathtt{x} \in \mathbf{IVar}_{\mathsf{S1}} \Rightarrow \ulcorner \varepsilon \mathtt{x} \mathtt{F} \urcorner \in \mathbf{Ter}_{\mathsf{S}}$.
5. $\mathtt{P} \in \mathbf{Pre}_{\mathtt{n}} \curlywedge \mathtt{t}_1, \ldots, \mathtt{t}_n \in \mathbf{Ter}_{\mathsf{S}} \Rightarrow \mathtt{P}\mathtt{t}_1 \ldots \mathtt{t}_n \in \mathbf{X} : n \in \mathbb{N}$.
6. $\mathtt{F} \in \mathbf{X} \curlywedge \mathtt{x} \in \mathbf{IVar}_{\mathsf{S1}} \Rightarrow \ulcorner \exists \mathtt{x}\, \mathtt{F} \urcorner, \ulcorner \forall \mathtt{x}\, \mathtt{F} \urcorner \in \mathbf{X}$.
7. $\mathbf{Fo}_{\mathsf{S}}$ is the intersection of all sets $\mathbf{X}$ satisfying 1–6.

As with $\mathbf{Fo}_{\mathsf{S0}}$, we may also present the formation rules more succinctly using $\overset{\text{DF}}{=}$.

Definition 10.14 (Formulae with indefinite descriptors)**.**

$$\mathbf{Ter}_{\mathsf{S}} \overset{\text{DF}}{=} \mathbf{ICon}_{\mathsf{S1}} \cup \mathbf{IVar}_{\mathsf{S1}} \cup \{\ulcorner \varepsilon \mathtt{x} \mathtt{F} \urcorner : \mathtt{F} \in \mathbf{Fo}_{\mathsf{S}} \curlywedge \mathtt{x} \in \mathbf{IVar}_{\mathsf{S1}}\}$$

$$\begin{aligned} \mathbf{Fo}_{\mathsf{S}} \overset{\text{DF}}{=} & \{\mathtt{P}\mathtt{t}_1 \ldots \mathtt{t}_\mathtt{n} : \mathtt{P} \in \mathbf{Pre}_{\mathsf{S}}[\mathtt{n}] \curlywedge \mathtt{t}_1 \ldots \mathtt{t}_\mathtt{n} \in \mathbf{Ter}_{\mathsf{S}}\} \cup \\ & \{\ulcorner \neg \mathtt{F} \urcorner : \mathtt{F} \in \mathbf{Fo}_{\mathsf{S}}\} \cup \\ & \{\ulcorner (\mathtt{F} \heartsuit \mathtt{G}) \urcorner : \mathtt{F}, \mathtt{G} \in \mathbf{Fo}_{\mathsf{S}} \curlywedge \heartsuit \in \mathbf{Con}_{\mathsf{S1}}[2]\} \cup \\ & \{\circledcirc \mathtt{x} \mathtt{F} : \circledcirc \in \mathbf{Qua}_{\mathsf{S1}} \curlywedge \mathtt{x} \in \mathbf{IVar}_{\mathsf{S1}} \curlywedge \mathtt{F} \in \mathbf{Fo}_{\mathsf{S}}\} \\ & : n \in \mathbb{N} \end{aligned}$$

An expression like '$\varepsilon x A(x)$' denotes some (not necessarily unique) x such that $A(x)$, if such an x exists; otherwise it denotes an arbitrary object in the domain.

Task 10.6. Provide natural language expressions with the structure of the following formulae with definite descriptors:

1. '$S^1 \varepsilon x P^1 x$'. **Regimented:** '$S^1(\varepsilon x P^1(x))$'.
2. '$S^1 \varepsilon x P^1 x$'. **Regimented:** '$S^1(\varepsilon x P^1(x))$'.
3. '$\neg P^1 \varepsilon x Q^2 x c$'. **Regimented:** '$\neg P^1(\varepsilon x Q^2(x, c))$'.
4. '$P^2 \varepsilon x Q^2 x a a$'. **Regimented:** '$P^2(\varepsilon x Q^2(x, a), a)$'.

Solution to task 10.6.

1. '$S^1 \varepsilon x P^1 x$'. **Regimented:** '$S^1(\varepsilon x P^1(x))$'.

   Natural language expression:

   Whoever is president is sly.
2. '$\neg Q^1 \varepsilon x S x$'. **Regimented:** '$\neg Q^1(\varepsilon x S(x))$'.

   Natural language expression:

   Whoever stole this is not the Queen.
3. '$\neg P^1 \varepsilon x Q^2 x c$'. **Regimented:** '$\neg P^1(\varepsilon x Q^2(x, c))$'.

   Natural language expression:

   Whoever witnessed the crime is not around.
4. '$P^2 \varepsilon x Q^2 x a a$'. **Regimented:** '$P^2(\varepsilon x Q^2(x, a), a)$'.

   Natural language expression:

   The person who called Andrew is after him.

There is a remarkable logical system that uses 'ϵ' and is commonly referred to as ϵ-calculus. In the ϵ-calculus, we usually do not have quantifiers as primitive symbols. We rather define the existential quantifier as follows:

$$\exists \mathrm{x}\, \mathrm{Px} \overset{\mathrm{DF}}{=} \mathrm{P}\epsilon\mathrm{xPx}$$

$$\mathrm{P}(\epsilon\mathrm{xP}(\mathrm{x})) \qquad \text{(regimented)}$$

$$: \mathrm{P} \in \mathbf{Pre}_1$$

Can you make intuitive sense of that?

10.3.3 The lambda abstractor

The lambda abstractor is an operator for forming complex predicate-terms from formulae. Intuitively, while the operators 'ι' and 'ε' form *singular terms* intended to denote objects, the lambda abstractor forms *predicate-terms* intended to denote properties or relations.

We introduce it in the form of expressions $\ulcorner \lambda \mathtt{x} \mathtt{F} \urcorner$. Such expressions are treated as predicate-expressions which can be applied to terms in the usual way.

As before, we work in the vocabulary of S1. Thus, we extend the language $\mathbf{Fo}_{S1}$ to a language $\mathbf{Fo}_{S}$ with the lambda-abstractor as follows.

Preliminary 10.15 (Formulae with the lambda abstractor)**.** Where $\mathtt{n} \in \mathbf{Num}_{\mathbb{W}}$, the set $\mathbf{Fo}_{S1}$ is recursively defined as follows:

1. $\mathtt{P} \in (\mathbf{Pre}_{\mathtt{n}} \cup \mathbf{Abs}_{\mathtt{n}}) \mathrel{\wedge\!\!\!\wedge} \mathtt{t}_1, \ldots, \mathtt{t}_n \in \mathbf{ITer}_{S1} \Rightarrow \mathtt{P}\mathtt{t}_1 \ldots \mathtt{t}_n \in \mathbf{X}$ $\quad : n \in \mathbb{N}.$
2. $\mathtt{F} \in \mathbf{X} \Rightarrow \ulcorner \neg \mathtt{F} \urcorner \in \mathbf{X}.$
3. $\mathtt{F}, \mathtt{G} \in \mathbf{X} \Rightarrow \ulcorner (\mathtt{F} \vee \mathtt{G}) \urcorner, \ulcorner (\mathtt{F} \wedge \mathtt{G}) \urcorner, \ulcorner (\mathtt{F} \rightarrow \mathtt{G}) \urcorner, \ulcorner (\mathtt{F} \leftrightarrow \mathtt{G}) \urcorner \in \mathbf{X}.$
4. $\mathtt{F} \in \mathbf{X} \mathrel{\wedge\!\!\!\wedge} \mathtt{x}_1, \ldots, \mathtt{x}_n \in \mathbf{IVar}_{S1} \Rightarrow \ulcorner \lambda \mathtt{x}_1 \ldots \mathtt{x}_{\mathtt{n}} \mathtt{F} \urcorner \in \mathbf{Abs}_{\mathtt{n}}.$
5. $\mathtt{F} \in \mathbf{X} \mathrel{\wedge\!\!\!\wedge} \mathtt{x} \in \mathbf{IVar}_{S1} \Rightarrow \ulcorner \exists \mathtt{x}\, \mathtt{F} \urcorner, \ulcorner \forall \mathtt{x}\, \mathtt{F} \urcorner \in \mathbf{X}.$
6. $\mathbf{Fo}_{S}$ is the intersection of all sets $\mathbf{X}$ satisfying 1–5.

As with ι- and ε-expressions, we may give a more compact definition.

Definition 10.16 (Formulae with the lambda abstractor)**.**

$$
\begin{aligned}
\mathbf{Abs}_{\mathtt{n}} &\overset{\text{DF}}{=} \{\ulcorner \lambda \mathtt{x}_1 \ldots \mathtt{x}_n \mathtt{F} \urcorner : \mathtt{F} \in \mathbf{X} \mathrel{\wedge\!\!\!\wedge} \mathtt{x}_1, \ldots, \mathtt{x}_n \in \mathbf{IVar}_{S1}\} \\
\mathbf{Fo}_{S} &\overset{\text{DF}}{=} \{\mathtt{P}\mathtt{t}_1 \ldots \mathtt{t}_n : \mathtt{P} \in (\mathbf{Pre}_{\mathtt{n}} \cup \mathbf{Abs}_{\mathtt{n}}) \mathrel{\wedge\!\!\!\wedge} \mathtt{t}_1 \ldots \mathtt{t}_n \in \mathbf{ITer}_{S1}\} \cup \\
&\quad \{\ulcorner \neg \mathtt{F} \urcorner : \mathtt{F} \in \mathbf{Fo}_{S}\} \cup \\
&\quad \{\ulcorner (\mathtt{F} \heartsuit \mathtt{G}) \urcorner : \mathtt{F}, \mathtt{G} \in \mathbf{Fo}_{S} \mathrel{\wedge\!\!\!\wedge} \heartsuit \in \mathbf{Con}_{S1}[2]\} \cup \\
&\quad \{\circ \mathtt{x} \mathtt{F} : \circ \in \mathbf{Qua}_{S1} \mathrel{\wedge\!\!\!\wedge} \mathtt{x} \in \mathbf{IVar}_{S1} \mathrel{\wedge\!\!\!\wedge} \mathtt{F} \in \mathbf{Fo}_{S}\} : n \in \mathbb{N}
\end{aligned}
$$

A *lambda abstract* is an expression of the form $\ulcorner \lambda \mathtt{x}_1 \ldots \mathtt{x}_n \mathtt{F} \urcorner$, which denotes the property of being an $\mathtt{x}$ such that $\mathtt{F}$ holds.

Task 10.7. Provide natural language expressions having the form of the following lambda abstracts:

1. '$\lambda x P^1 x$'. **Regimented:** '$\lambda x P^1(x)$'.
2. '$\lambda x \neg P^1 x$'. **Regimented:** '$\lambda x \neg P^1(x)$'.
3. '$\lambda x Q^2 x a$'. **Regimented:** '$\lambda x Q^2(x, a)$'.
4. '$\lambda x Q^2 a x$'. **Regimented:** '$\lambda x Q^2(a, x)$'.
5. '$\lambda x (P^1 x \wedge Q^1 x)$'. **Regimented:** '$\lambda x (P^1(x) \wedge Q^1(x))$'.

Solution to task 10.7.

1. '$\lambda x P^1 x$'. **Regimented:** '$\lambda x P^1(x)$'.

   Natural language expression:

   The property of being a president.
2. '$\lambda x \neg P^1 x$'. **Regimented:** '$\lambda x \neg P^1(x)$'.

   Natural language expression:

   The property of not being a president.
3. '$\lambda x Q^2 x a$'. **Regimented:** '$\lambda x Q^2(x, a)$'.

   Natural language expression:

   The property of loving Andrew.
4. '$\lambda x Q^2 a x$'. **Regimented:** '$\lambda x Q^2(a, x)$'.

   Natural language expression:

   The property of being loved by Andrew.
5. '$\lambda x (P^1 x \wedge Q^1 x)$'. **Regimented:** '$\lambda x (P^1(x) \wedge Q^1(x))$'.

   Natural language expression:

   The property of being both a philosopher and a mathematician.

Lambda abstraction allows us to apply complex predicates to a singular term as if it were a single predicate.

Task 10.8. Formalise the following natural language expressions using the corresponding lambda abstracts used in task 10.7:

1. Andrew is president.
2. Andrew is not a president.
3. Bertha loves Andrew.
4. Andrew loves Bertha.
5. Bertha is a philosopher and a mathematician.

Solution to task 10.8.

1. Andrew is president.

   Representation: '$\lambda x P^1 x a$'.

   Regimented: '$(\lambda x P^1(x))(a)$'.

2. Andrew is not a president.

   Representation: '$\lambda x \neg P^1 x a$'.

   Regimented: '$(\lambda x \neg P^1(x))(a)$'.

3. Bertha loves Andrew.

   Representation: '$\lambda x Q^2 x a b$'.

   Regimented: '$(\lambda x Q^2(x, a))(b)$'.

4. Andrew loves Bertha.

   Representation: '$\lambda x Q^2 a x b$'.

   Regimented: '$(\lambda x Q^2(a, x))(b)$'.

5. Bertha is a philosopher and a mathematician.

   Representation: '$\lambda x (P^1 x \wedge Q^1 x) b$'.

   Regimented: '$\lambda x (P^1(x) \wedge Q^1(x))(b)$'.

11 Formula length

The length of the expressions of a formal language may also be a criterion for classifying formal language. The concept of the length of an expression e, or $|\mathsf{e}|$, may be defined as follows.

Definition 11.1 (Length of an expression)**.**

$$|\mathsf{c}| \overset{\text{DF}}{=} 1$$
$$|\mathsf{ef}| \overset{\text{DF}}{=} |\mathsf{e}| + |\mathsf{f}|$$
$$: \mathsf{c} \text{ is a character.}$$

Note the following corollary:

Corollary 11.2. $|‘’| \overset{\text{DF}}{=} 0.$

Although expressions in principle have a length equal to some natural language, it is possible to think about expressions consisting of an infinitely long chain of characters. Think, e.g., of the number π expressed in decimal notation.

Based on this, we may distinguish between languages which only admit finitely long formulae (*finitary languages*) and languages which also admit infinitely long formulae (i.e., *infinitary languages*).

Definition 11.3 (Finitary language)**.** A formal language $\mathbf{Fo}_\mathsf{S}$ is **finitary** $\overset{\text{DF}}{=} \forall \mathsf{e} \in \mathbf{Fo}_\mathsf{S}\, \exists n \in \mathbb{N} : |\mathsf{e}| \leq n$.

Definition 11.4 (Infinitary language)**.** A formal language $\mathbf{Fo}_\mathsf{S}$ is **infinitary** $\overset{\text{DF}}{=} \exists \mathsf{e} \in \mathbf{Fo}_\mathsf{S}\, \forall n \in \mathbb{N} : |\mathsf{e}| > n$.

Most languages studied in this textbook, in particular $\mathbf{Fo}_{\mathsf{S0}}$, $\mathbf{Fo}_{\mathsf{S1}}$, $\mathbf{Fo}_\mathsf{A}$, are finitary. Among finitary languages, we might distinguish between languages with formulae of limited length (which is the case of $\mathbf{Fo}_\mathsf{A}$, all of whose formulae have length 3), and of unlimited or arbitrary length (which is the case of all modern languages which are finitary).

On the other hand, infinitary logics admit infinitely long formulae. Among such logics we find, for example, logics with infinitary relations, i.e., relations of infinite arity. In the case of infinitary languages, it is not possible to write down their formulae of infinite length, but it is possible to formulate certain rules of inference that apply to them.

Some may think that standard S1 is an infinitary logic in disguise. Consider the formula ‘$\forall x\, Px$’ as an abbreviation of the following infinitely long formula:

$$P\mathtt{a}_1 \wedge P\mathtt{a}_2 \wedge \ldots,$$

where each $\mathtt{a}_i$ is an individual constant of our language, and all individual constants of our language correspond to some $\mathtt{a}_i$.

Despite the possible equivalence between those formulae in an infinitary logic, '$\forall x\, Px$' itself is a formula of finite length.

Infinitary logics break the condition that a string should be a finite sequence of characters. However, these infinitely long formulae are usually handled via a finite shorthand as follows.

Let **A** be an infinite set of formulae. Their infinitary conjunction and disjunction may be abbreviated as follows:

$$\bigwedge_{\mathtt{A}_i \in \mathbf{A}} \mathtt{A}_i \stackrel{\text{DF}}{=} \mathtt{A}_1 \wedge \mathtt{A}_2 \wedge \ldots \qquad \text{(Infinitary conjunction)}$$

$$\bigvee_{\mathtt{A}_i \in \mathbf{A}} \mathtt{A}_i \stackrel{\text{DF}}{=} \mathtt{A}_1 \vee \mathtt{A}_2 \vee \ldots \qquad \text{(Infinitary disjunction)}$$

Some infinitary formulae are nevertheless very difficult to handle in this way.

12 Kind of Quantification

In this chapter, we consider formal languages with various kinds of quantification. Thus, these are formal languages which must be of first-order or higher order.

12.1 Sort

A sort is a set of objects or entities that share the same type within a logical system. It may be understood as a category that groups together elements with similar properties, allowing us to distinguish between different kinds of objects when reasoning or quantifying in a logical language.

This notion organises the logical domain by ensuring that variables and predicates apply only to certain types of objects, according to their sort.

In S1 there is only one sort, which encompasses all objects in the domain of reference. This means that no distinction is made between different kinds of entities: variables and predicates apply uniformly to all objects in the domain, regardless of their particular characteristics. For this reason, S1 is said to be one-sorted (monosorted).

By contrast, many-sorted logics allow for multiple sorts, that is, different domains of objects. Thus, we may have one set of variables quantifying over numbers, another over persons, and so on, thereby permitting more specific relations between different kinds of objects within a single logical system.

In what follows, we will introduce a many sorted language MS, which is an approximation to how these languages may be defined (see SEP).

In order to construct MS, we need to assign each of our singular terms a sort. To do so, we need sort terms.

Let us use the set:

$$\mathbf{Stu} \overset{\text{DF}}{=} \{\text{‘}s\text{’}, \text{‘}t\text{’}, \text{‘}u\text{’}\}$$

as our set of sort letters. Thus, we might define our set $\mathbf{STer}_{\mathsf{MS}}$ of sort terms as follows.

Definition 12.1 (Sort terms of MS)**.**

$$\mathbf{STer}_{\mathsf{MS}} \overset{\text{DF}}{=} \{\mathtt{s}_{\mathtt{i}} : \mathtt{s} \in \mathbf{Stu} \wedge \mathtt{i} \in \mathbf{Num}_{\mathbb{N}}\}$$

Hence, the expressions 's', 's_1', '...', 't', 't_1', '...', 'u', 'u_1', '...' will be sort terms. If we want just a two sorted language, we just might define: $\mathbf{STer}_{\mathsf{MS}} \overset{\text{DF}}{=} \{\text{'}s\text{'}, \text{'}t\text{'}\}$.

Now, in order to mark that a singular term is of a sort called 's', we will write 's' on top of a variable or constant which belongs to that sort. Thus, our individual terms will be defined as follows.

Definition 12.2 (Individual terms of MS)**.**

$$\mathbf{ICon}_{\mathsf{MS}} \overset{\text{DF}}{=} \{\overset{\mathsf{s}}{\mathsf{a}} : \mathsf{a} \in \mathbf{ICon}_{\mathsf{S1}} \wedge \mathsf{s} \in \mathbf{STer}_{\mathsf{MS}}\} \quad (\Upsilon)$$
$$\mathbf{IVar}_{\mathsf{MS}} \overset{\text{DF}}{=} \{\overset{\mathsf{s}}{\mathsf{x}} : \mathsf{x} \in \mathbf{IVar}_{\mathsf{S1}} \wedge \mathsf{s} \in \mathbf{STer}_{\mathsf{MS}}\} \quad (\forall)$$
$$\mathbf{ITer}_{\mathsf{MS}} \overset{\text{DF}}{=} \mathbf{ICon}_{\mathsf{MS}} \cup \mathbf{IVar}_{\mathsf{MS}} \quad (\text{II})$$

Hence, the expressions '$\overset{s}{x}$', '$\overset{s}{x}_1$', '...', '$\overset{s}{y}$', '$\overset{s}{y}_1$', '$\overset{s}{z}$', '$\overset{s}{z}_1$' are variables of the sort s; the expressions '$\overset{s_1}{x}$', '$\overset{s_1}{x}_1$', '...', '$\overset{s_1}{y}$', '$\overset{s_1}{y}_1$', '$\overset{s_1}{z}$', '$\overset{s_1}{z}_1$' are variables of the sort s_1; etc. Similarly, the expressions '$\overset{s}{a}$', '$\overset{s}{a}_1$', '...', '$\overset{s}{b}$', '$\overset{s}{b}_1$', '$\overset{s}{c}$', '$\overset{s}{c}_1$' are constants of the sort s; the expressions '$\overset{s_1}{a}$', '$\overset{s_1}{a}_1$', '...', '$\overset{s_1}{b}$', '$\overset{s_1}{b}_1$', '$\overset{s_1}{c}$', '$\overset{s_1}{c}_1$' are constants of the sort s_1; etc.

Note here that the expressions '$\overset{s}{a}$' and '$\overset{t}{a}$' are two distinct singular terms, to which different denotations may be assigned – especially if 's' and 't' stand for distinct sorts.

Now we need to define our predicates. Note that some predicates are only applicable to some sorts. For instance, 'is green' cannot be said of a number, like 2.

Moreover, some relators (predicates of two or more places) require arguments of various sorts. For instance, 'reads' relates a person (in the left) to a text (in the left). In order to formalise this idea, we introduce the notion of a predicate's rank, represented by $\ulcorner\mathsf{Rank}\mathtt{P}\urcorner$, where $\mathtt{P}$ is a predicate.

Definition 12.3 (Rank of a predicate)**.**

$$\begin{aligned}\mathsf{Rank}(\mathtt{P}) &\overset{\text{DF}}{=} \langle \mathbf{S}_1, \ldots, \mathbf{S}_n \rangle \\ &\quad : \mathbf{S}_1, \ldots, \mathbf{S}_n \subseteq \mathbf{STer}_{\mathsf{MS}} \mathbin{\wedge\!\!\!\wedge} \mathtt{P} \in \mathbf{Pre}_n.\end{aligned}$$

Intuitively, $\mathsf{Rank}(\mathtt{P})[i]$ is the set of sort terms which can occur in the i-th argument of the n-place predicate $\mathtt{P}$ – where $i \leq n$. In other words, $\mathsf{Rank}(\mathtt{P})$ contains the ways in which an atomic formula with $\mathtt{P}$ can be constructed.

With tall these notions defined, the formal language $\mathbf{Fo}_{\mathsf{MS}}$ may be recursively defined as follows:

Preliminary 12.4 (Formulae of $\mathbf{Fo}_{\mathsf{MS}}$). Where $\mathtt{n} \in \mathbf{Num}_{\mathbb{N}}$, the set $\mathbf{Fo}_{\mathsf{MS}}$ is recursively defined as follows:

1. $\mathtt{P} \in \mathbf{Pre}_{\mathtt{n}} \mathrel{\&} \overset{\mathtt{s}_1}{\mathtt{t}}_1, \ldots, \overset{\mathtt{s}_n}{\mathtt{t}}_n \in \mathbf{ITer}_{\mathsf{MS}} \mathrel{\&} \mathsf{Rank}(\mathtt{P}) = \langle \mathbf{S}_1, \ldots, \mathbf{S}_n \rangle \mathrel{\&} \mathtt{s}_1 \in \mathbf{S}_1, \ldots, \mathtt{s}_n \in \mathbf{S}_n \Rightarrow \mathtt{P}\overset{\mathtt{s}_1}{\mathtt{t}}_1 \ldots \overset{\mathtt{s}_n}{\mathtt{t}}_n \in \mathbf{X}$:.
2. $\mathtt{F} \in \mathbf{X} \Rightarrow \ulcorner\neg\mathtt{F}\urcorner \in \mathbf{X}$.
3. $\mathtt{F}, \mathtt{G} \in \mathbf{X} \Rightarrow \ulcorner(\mathtt{F} \vee \mathtt{G})\urcorner, \ulcorner(\mathtt{F} \wedge \mathtt{G})\urcorner, \ulcorner(\mathtt{F} \rightarrow \mathtt{G})\urcorner, \ulcorner(\mathtt{F} \leftrightarrow \mathtt{G})\urcorner \in \mathbf{X}$.
4. $\mathtt{F} \in \mathbf{X} \mathrel{\&} \mathtt{x} \in \mathbf{IVar}_{\mathsf{MS}} \Rightarrow \ulcorner\exists\mathtt{x}\,\mathtt{F}\urcorner, \ulcorner\forall\mathtt{x}\,\mathtt{F}\urcorner \in \mathbf{X}$.
5. $\mathbf{Fo}_{\mathsf{MS}}$ is the intersection of all sets $\mathbf{X}$ satisfying 1–4.

As with '$\mathbf{Fo}_{\mathsf{S0}}$', we might define '$\mathbf{Fo}_{\mathsf{MS}}$' using '$\overset{\text{DF}}{=}$'.

Task 12.1. Define '$\mathbf{Fo}_{\mathsf{MS}}$' using '$\overset{\text{DF}}{=}$'.

Definition 12.5 (Formulae of MS (answer to task 12.1)).

$$\begin{aligned}\mathbf{Fo}_{\mathsf{MS}} \overset{\text{DF}}{=} \{&\mathtt{P}\overset{\mathtt{s}_1}{\mathtt{t}}_1 \ldots \overset{\mathtt{s}_n}{\mathtt{t}}_n : \mathtt{P} \in \mathbf{Pre}_{\mathtt{n}} \mathrel{\&} \overset{\mathtt{s}_1}{\mathtt{t}}_1, \ldots, \overset{\mathtt{s}_n}{\mathtt{t}}_n \in \mathbf{ITer}_{\mathsf{MS}} \mathrel{\&} \\ &\mathsf{Rank}(\mathtt{P}) = \langle \mathbf{S}_1, \ldots, \mathbf{S}_n \rangle \mathrel{\&} \mathtt{s}_1 \in \mathbf{S}_1, \ldots, \mathtt{s}_n \in \mathbf{S}_n\} \cup \\ &\{\ulcorner\neg\mathtt{F}\urcorner : \mathtt{F} \in \mathbf{Fo}_{\mathsf{MS}}\} \cup \\ &\{\ulcorner(\mathtt{F} \heartsuit \mathtt{G})\urcorner : \mathtt{F}, \mathtt{G} \in \mathbf{Fo}_{\mathsf{MS}} \mathrel{\&} \heartsuit \in \mathbf{Con}_{\mathsf{MS}}[2]\} \cup \\ &\{\circ\mathtt{vF} : \circ \in \mathbf{Qua}_{\mathsf{MS}} \mathrel{\&} \mathtt{x} \in \mathbf{IVar}_{\mathsf{MS}} \mathrel{\&} \mathtt{F} \in \mathbf{Fo}_{\mathsf{MS}}\} \\ &: \mathtt{n} \in \mathbf{Num}_{\mathbb{N}}\end{aligned}$$

This defines a multi-sorted language.

Task 12.2. Formalise the following expressions using the sort terms '*s*' for books, '*t*' for courses, and '*u*' for persons. (You may assume that your predicate is of the appropriate rank.)

1. Every person has read some book.
2. Some student is enrolled in all courses.
3. Every book has at least one author.
4. Some course requires no book.

Solution to task 12.2.

1. Every person has read some book.
   Representation: '$\forall \overset{u}{x}\ \exists \overset{s}{y}\ R\overset{u}{x}\overset{s}{y}$'.
2. Some student is enrolled in all courses.
   Representation: '$\exists \overset{u}{x}\ \forall \overset{t}{y}\ E\overset{u}{x}\overset{t}{y}$'.
3. Every book has at least one author.
   Representation: '$\forall \overset{s}{x}\ \exists \overset{u}{y}\ A\overset{u}{y}\overset{s}{x}$'.
4. Some course requires no book.
   Representation: '$\exists \overset{t}{x}\ \neg\exists \overset{s}{y}\ R\overset{t}{x}\overset{s}{y}$'.

12.2 Quantificational singularity and plurality

We may also distinguish between quantificationally singular languages and quantificationally plural languages. This classification reflects how each language handles reference to, and reasoning about, individual objects as opposed to groups of objects.

Quantificationally singular languages, such as classical logic, focus primarily on individual entities. In these languages, quantification is carried out using singular terms and predicates, which allows us to make statements only about individual objects. For example, we may say:

Sentence 12.6. For every student, there is a book.
Representation: $\forall x\,(E^1 x \rightarrow \exists y\; B^2 y x)$.

Here, the quantifiers bind individual variables ('x' and 'y'), indicating that each instance is treated separately. The logic is structured around predicates applied to singular terms, and reasoning typically revolves around individual objects rather than collections.

By contrast, quantificationally plural languages, such as plural logics, are designed to handle multiple objects simultaneously. These languages employ plural terms and plural quantifiers, which make it possible to formulate statements about groups or collections without referring to their individual members. For example, a plural logic might express:

Sentence 12.7. Some students are celebrating.
Representation: $\exists \underline{xx}\,(E^1 \underline{xx} \wedge C^1 \underline{xx})$.

In this example, the plural quantifier '$\exists \underline{xx}$' allows us to refer to a group of students collectively. Plural logics facilitate reasoning about properties that apply to a group as a whole, without reducing such reasoning to claims about each individual.

12.3 Order

We can distinguish between zero-order, first-order, second-order systems, and so on. This is related to the scope of the quantifier.

For example, in propositional logics we do not have quantifiers, so they are of zero order.

Predicate logics, by contrast, have quantifiers whose scope ranges over individual variables; therefore, they are first-order logics.

Metalogical systems normally have quantifiers whose scope ranges over predicates typical of first-order logic, and thus they are second-order logics.

Aristotelian syllogistic is a difficult case to classify. One could argue that it is of second order, since its quantifiers bind terms which refer to categories, that is, they bind predicates. However, because it has no individual variables, such predicates do not apply to variables and do not satisfy the conditions of a second-order logic.

Since Aristotle's syllogistic can be reconstructed as a fragment of first-order logic, it may be more appropriate to say that it is a first-order logic.

Task 12.3. Formalise the following expressions using second order quantifiers.

1. Everything that is true is the case. (You may use '**1**' here.)
2. Everything that is necessary is possible. (You may alethic modal operators here.)
3. Socrates has at least one property.
4. Someone has at least one property.

Solution to task 12.3.

1. Everything that is true is the case.
   Representation: '$\forall^2 p\,(p = \mathbf{1} \rightarrow p)$'.
2. Everything that is necessary is possible.
   Representation: '$\forall^2 p\,(\Box p \rightarrow \Diamond p)$'.
3. Socrates has at least one property.
   Representation: '$\exists^2 P\; Ps$'.
4. Someone has at least one property.
   Representation: '$\exists^1 x\; \exists^2 P\; Px$'.